Interpreter

Interpreter · Pricing-rule DSL: base + percent + flat discounts · Build with Waqar

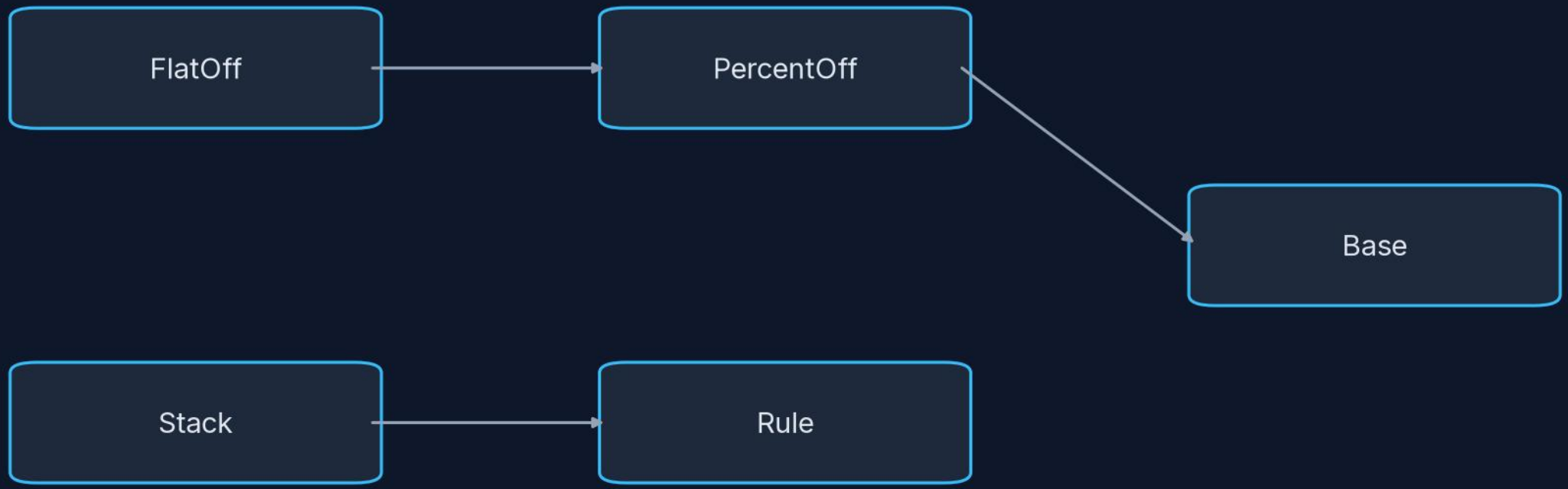
Today's Hook

- Problem: Pricing-rule DSL: base + percent + flat discounts
- Pattern: Interpreter — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Pricing-rule DSL: base + percent + flat discounts
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Interpreter: Pricing-rule DSL: base + percent + flat discounts

Real-World Scenario

- Running example: Pricing-rule DSL: base + percent + flat discounts
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

/** Naive: stringly if/else discount codes — untestable composition. */
public final class NaivePricing {
    public static int price(String code, Map<String, Integer> prices) {
        int base = prices.getOrDefault("SKU", 0);
        if ("PCT10".equals(code)) return base * 90 / 100;
        if ("FLAT200".equals(code)) return Math.max(0, base - 200);
        if ("STACK".equals(code)) return Math.max(0, base * 90 / 100 - 200);
        throw new IllegalArgumentException("unknown " + code);
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

/** Pricing-rule DSL: Base / PercentOff / FlatOff / Stack as an Expression AST. */
public final class PricingDsl {
    private PricingDsl() {}
    public sealed interface Rule permits Base, PercentOff, FlatOff, Stack {
        int apply(Map<String, Integer> ctx);
    }
    public record Base(String skuKey) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            Integer v = ctx.get(skuKey);
            if (v == null) throw new IllegalArgumentException("unknown sku " + skuKey);
            return v;
        }
    }
    public record PercentOff(Rule inner, int percent) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            if (percent < 0 || percent > 100) throw new IllegalArgumentException("bad percent");
            return inner.apply(ctx) * (100 - percent) / 100;
        }
```

Solution (2)

```
    }
    public record FlatOff(Rule inner, int cents) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            return Math.max(0, inner.apply(ctx) - cents);
        }
    }
    public record Stack(Rule left, Rule right) implements Rule {
        public int apply(Map<String, Integer> ctx) { return left.apply(ctx) + right.apply(ctx); }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Interpreter: pricing-rule DSL ===");
        var ctx = Map.of("SKU", 2000, "ADDON", 500);
        System.out.println("--- naive ---");
        System.out.println(NaivePricing.price("STACK", ctx));
        System.out.println("--- DSL AST ---");
        PricingDsl.Rule rule = new PricingDsl.FlatOff(
                new PricingDsl.PercentOff(new PricingDsl.Base("SKU"), 10), 200);
        System.out.println("price=" + rule.apply(ctx));
        PricingDsl.Rule stack = new PricingDsl.Stack(rule, new PricingDsl.Base("ADDON"));
        System.out.println("cart=" + stack.apply(ctx));
        try {
            new PricingDsl.Base("NOPE").apply(ctx);
        } catch (IllegalArgumentException e) {
            System.out.println("caught=" + e.getMessage());
        }
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Interpreter: pricing-rule DSL ===
--- naive ---
1600
--- DSL AST ---
price=1600
cart=2100
caught=unknown sku NOPE
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Interpreter.

Related Patterns

- Contrast Interpreter with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Interpreter → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.